软件需求分析与设计

第二章框架梳理

仅供学习参考

目 录

一、 从业务过程到解决方案的构想 - 1 -

（一） 过程层次建模（Process Hierarchy Modeling） - 1 -

1. 业务过程模型 - 1 -

（二） 业务过程建模 - 1 -

1. 业务过程建模表示法（Business Process Modeling Notation，BPMN） - 1 -

（三） 解决方案构想（Solution Envisioning） - 2 -

1. 业务价值驱动方法，以提供解决当前业务问题、促进将来业务创新的 IT 服务 - 2 -

2. 在业务和 IT 利益相关者之间建立了紧密的联系，并且整合了业务战略方法和软件开发能力 - 2 -

3. 实现的支撑（3E） - 2 -

4. 解决方案构想的三个阶段 - 2 -

5. 实现策略和能力体系结构 - 2 -

二、 需求引导（Elicitation）——最不需要技术，但最需要技巧的环节 - 3 -

（一） 功能性需求 - 3 -

1. 由系统期望的服务（服务陈述） - 3 -

（二） 非功能性需求（补充需求） - 3 -

1. 系统必须遵守的约束（约束陈述），如性能、安全性 - 3 -

2. 七种非功能性需求 - 3 -

（三） 需求引导的方法 - 3 -

1. 传统方法——适用于目标明确，低风险的项目 - 3 -

2. 现代方法——适用于风险高的项目 - 4 -

三、 需求协商与确认（Negotiation & Validation） - 6 -

（一） 需求协商与确认的关系 - 6 -

1. 需求协商与确认的必要性： - 6 -

2. 需求协商与确认&需求引导同步进行 - 6 -

3. 需求协商以文档的草稿为基础，需求确认需要更加完整的文档确认版本 - 6 -

（二） 超出范围的需求 - 6 -

1. 需要对照参考模型才能决定 - 6 -

（三） 需求依赖矩阵（交互矩阵） - 6 -

1. 需求依赖矩阵的特点 - 6 -

（四） 需求风险和优先级 - 6 -

仅供学习参考

1. 风险分类 - 6 -
2. 风险数量决定了项目的可行性（Feasibility），需要对风险进行分析和优先级排序 - 6 -
四、 需求管理 - 7 -
（一） 需求标识与分类 - 7 -
1. 需求以自然语言描述 - 7 -
2. 对需求进行标识分类的 3 种技术 - 7 -
（二） 需求层次 - 7 -
1. 按父子关系建立层次化结构 - 7 -
（三） 变更管理 - 7 -
1. 需求变更的原因 - 7 -
2. 需求变更的实现 - 7 -
五、 需求业务模型 - 8 -
（一） 需求业务模型的定义 - 8 -
1. 在需求确定阶段，所收集需求的高层可视化表示 - 8 -
2. 不属于 UML，是高层可视化模型 - 8 -
3. UML 进行形式化的需求建模在需求规格说明阶段进行 - 8 -
（二） 系统范围模型 - 8 -
1. 系统范围的作用 - 8 -
2. 界定系统范围的方法 - 8 -
（三） 业务用例模型 - 8 -
1. 特点 - 8 -
2. 业务用例 - 8 -
3. 业务用例图 - 8 -
4. 业务词汇表 - 9 -
5. 业务类模型 - 9 -
六、 需求文档 - 10 -
（一） 项目准备 - 10 -
1. 项目的目的和范围 - 10 -
2. 业务环境 - 10 -
3. 利益相关者（Stakeholders） - 10 -
4. 多种解决方案 - 10 -
5. 文档综述 - 10 -
（二） 系统服务（功能需求）——占总篇幅的一半及以上 - 10 -

仅供学习参考

1. 系统范围——使用环境图建模 - 10 -
2. 功能性需求——使用业务用例图建模 - 10 -
3. 数据需求——使用业务类图建模 - 10 -
（三） 系统约束（非功能性需求） - 10 -
1. 界面需求——大概画出界面的外观，给一个界面原型 - 10 -
2. 性能需求，如响应时间、可靠性、吞吐量，用可测量可衡量的数据表示 - 10 -
3. 安全性需求，访问权限 - 10 -
4. 操作性需求，决定系统运行的软硬件环境 - 10 -
5. 政策法律需求 - 10 -
6. 其他约束 - 10 -
（四） 项目其他问题 - 10 -
1. 开放问题 - 10 -
2. 初步安排 - 10 -
3. 初步预算——初步安排的直接结果 - 10 -
（五） 附录 - 10 -
1. 业务词汇表（术语、简写词） - 10 -
2. 业务文档和表格 - 10 -
3. 参考文献 - 10 -

仅供学习参考

一、从业务过程到解决方案的构想

（一）过程层次建模（Process Hierarchy Modeling）

1. 业务过程模型

(1) 过程是分层的，过程可能包含其他过程（子过程），将过程中的原子活动称为任务

(2) 多个不同大小的过程来定义，从全企业范围的过程到单独一个人完成的过程

(3) 过去：数据流图，作为一种复杂的功能分解技术

(4) 现在：**过程层次图**——不是 BPMN 的一部分

① 定义了业务过程模型的静态数据结构

② 它显示过程的层次结构，将顶层业务过程分解为子过程

(5) 过程和过程分解

① 一个业务过程能被看作手工操作的活动或者自动化服务

② 一个过程至少一个输入流和一个输出流

③ 一个过程可以是原子的和复合的

（二）业务过程建模

1. 业务过程建模表示法（Business Process Modeling Notation，BPMN）

(1) 专门用于对由活动定义的业务过程建模，活动能够产生对企业或其外部利益相关者有价值的事物

(2) 描述过程内部细节，目的是为业务人员和 IT 人士提供一种共同的沟通语言

(3) 四种基本建模元素

① 流对象（Flow Object）——BPMN 的核心元素

1) 事件（Events）——某些发生的事物，表示方式：O

2) 活动（Activity）——某些必须进行的工作，表示方式：▭

a. 下边界上有一个“+”，表示其为一个复合活动。复合活动可分解为一组子活动，被看作是展开的子活动。其他符号决定了其他属性：循环或者多实例执行

3) 路由（Gateway）——控制多个序列流的分支和聚合

② 连接对象（Connecting Object）

1) 序列流——表示一个过程中活动完成的序列。直线箭头：---------→

2) **消息流**——表示两个业务实体之间的消息（数据）流向。带圈虚线箭头：O - - - - →

3) 关联——关联两个流对象。虚线：- - - -

③ 泳池 / 泳道（Pools / Swim-Lanes）

1) 描述单一的参与者（实体）的活动

2) 分水平和竖直型，取决于作图的方便程度

④ 人工制品

仅供学习参考

1) 数据对象：活动需要的数据或者活动产生的数据。对于过程的消息流或者序列流，他们不会产生任何直接影响，但会提供额外的关于活动的信息。

2) 组：目的是文档或分析

3) 注释：为读者提供附加的文本信息。

（三）解决方案构想（Solution Envisioning）

1. 业务价值驱动方法，以提供解决当前业务问题、促进将来业务创新的 IT 服务
2. 在业务和 IT 利益相关者之间建立了紧密的联系，并且整合了业务战略方法和软件开发能力
3. 实现的支撑（3E）
   (1) 优势（Edge）
   (2) 效率（Efficiency）
   (3) 效果（Effectiveness）
4. 解决方案构想的三个阶段
   (1) 业务能力探索
      ① 业务能力——为企业的 IT 解决方案提交具体成果的能力
      ② 该阶段描述一个**能力案例（Capability case）**（确定一项功能商业价值的建模元素），即提供了解决方案思路，为每个能力生成一个业务案例
   (2) 解决方案能力构想
      ① 目的：将能力用例发展成解决方案概念，确保利益相关者对这个方案的意见一致。解决方案概念将业务环境作为输入，产生的未来新工作方法的构想作为输出
   (3) 软件能力设计
      ① 该阶段开发软件能力体系结构（Capability Architecture），是软件建模的一项活动，标识高层系统功能性构件和它们之间相互作用的模块
5. 实现策略和能力体系结构
   (1) 三种实现策略：
      ① 常规（Custom）开发——手工、单机、从零开发软件
      ② 基于包（Package-based）的开发——定制先前存在的软件包
      ③ 基于构件（Component-based）的开发（搭积木）——通过集成多个软件构件

仅供学习参考

二、需求引导（Elicitation）——最不需要技术，但最需要技巧的环节

（一）功能性需求

1. 由系统期望的服务（服务陈述）

(1) 3 组功能性需求

① 系统的范围（Scope）

② 必要的业务功能——功能性需求

③ 所需的数据结构——数据需求

(2) 功能性需求需要从客户处获得

(3) 收集到的需求需要仔细的分析以消除重叠和矛盾，这个过程总会导致需求评审和与客户的再次协商

（二）非功能性需求（补充需求）

1. 系统必须遵守的约束（约束陈述），如性能、安全性

2. 七种非功能性需求

(1) 可用性（Usability）：定义使用系统的容易程度，如适当的提示，错误处理，文档帮助

(2) 可复用性（Re-usability）：在新系统开发中，重复使用之前软件系统构件的容易程度

(3) 可靠性（Reliability）:与系统失败的频率和严重性以及系统从失效中恢复的程度相关

(4) 性能（Performance）：通过系统响应时间、事务处理时间、资源开销、并发用户数量等的期望确定

(5) 效率（Efficiency）：考虑时间效率和成本效率

(6) 适应性 / 可支持性（Supportability / Adaptiveness）：可理解性 + 可维护性 + 可扩展性

(7) 其他约束：政策法律约束，便捷性或可交互性等要求（包括对接口的约束）

（三）需求引导的方法

1. 传统方法——适用于目标明确，低风险的项目

(1) 面谈（Interviewing）——发现事实和聚集信息的基本技术

① 结构化访谈（正式的）

1) 需要提前准备，有一个明确的日程，预先确定问题

2) 需要非结构化面谈进行补充

3) 访谈的两类问题

a. 开放问题（Open-ended）：客户的答案无法估计

b. 封闭问题（Close-ended）：客户提供简单回答（是或否）

② 非结构化访谈（非正式的）

1) 没有预定的问题和预计目的，鼓励客户说出自己的想法，如街头采访

(2) 调查表（Questionnaires）

① 向客户收集信息的有效方法，一般用来作为面谈的补充形式，而非替代

仅供学习参考

② 调查表设计应该使回答问题变得更容易，尽量使用封闭式问题

③ 封闭式问题的三种形式

1) 多项选择问题（Multiple-choice）

2) 评价问题（Rating questions）（完全同意、反对）

3) 排序问题（Ranking question）

④ 当需要调查许多人的观点而他们又地理分散的时候，调查表特别有用和经济。

(3) 观察（Observation）

① 有效的发现事实的技术，获取面谈与调查表之外的获取不到的信息

② 3 种观察形式

1) 被动观察（Passive）：业务分析员观察业务活动而不直接打扰或干预，比如安装摄像机

2) 主动观察（Active）：业务分析员参与到活动中，成为团队的一部分

3) 解释观察（Explanatory）：工作时，用户向观察者说明他进行的活动

③ 观察应持续较长（prolonged）的一段时间，在不同的时间段上和不同的工作负荷下进行

(4) 研究业务文档

① 发现用例需求和领域知识需求的宝贵技术，经常被使用

2. 现代方法——适用于风险高的项目

(1) 原型法（Prototyping）

① 最常使用的现代需求引导方法

② 原型：演示系统→‘Quick and dirty’ solution to obtain feedback

③ 两种原型：

1) 丢弃（Throw-away）：需求引导完成时，该原型将被丢弃。主要集中在理解的最少的需求

2) 进化（Evolutionary）：需求引导过程之后，仍被保留并用来产生最终产品。将产品发布的速度作为目标，主要集中在理解的很好的需求上

(2) 头脑风暴（Brainstorming）

① 通过放下一系列约束来产生新思想或发现专业问题解决方案的一种会议技术

② 头脑风暴通过触发式问题的想法发挥作用，其他需求引导方法都不是

③ 调解人（Facilitator）：会议的主持人，为将产生的新想法界定问题/机会领域，称作问题机会陈述（Probortunity = problem + opportunity）

④ 会议最后一步，投票表决想法的优先顺序

(3) 联合应用开发（Joint Application Development，JAD）

① 类似于头脑风暴技术，一次会议可能需要几小时、几天或几周，参与人数 25-30 人

② 会议参加者

1) 领导 Leader

仅供学习参考

2) 文书 Scribe

3) 客户 Customer（用户和经理）

4) 开发人员 Developer

③ 利用了群体动力优势

(4) 快速应用开发（Rapid Application Development，RAD）

① 不仅是一种需求引导方法，还是将软件开发作为一种过程的方法

② 目标：快速交付系统解决方案，技术精良程度次要

③ 带来相应问题

1) 不一致的 GUI 设计

2) 支持软件复用的专业解决方案，非通用解决方案

3) 不足的文档

4) 难以维护和扩展的软件

④ 5 种技术

1) 进化原型（Evolutionary prototyping）

2) CASE 工具

3) 拥有先进工具的专业人员（Specialists With Advanced Tools，SWAT）

4) 交互式 JAD（Interactive JAD）

5) 时间盒（Time-boxing）

仅供学习参考

三、需求协商与确认（Negotiation & Validation）

（一）需求协商与确认的关系

1. 需求协商与确认的必要性：

   (1) 获取的需求可能是重叠的或者矛盾的（Overlap or Conflict）

   (2) 获取的需求可能是模棱两可的或者不实际的（Ambiguous or Unrealistic）

   (3) 获取的需求可能是没有发现的（Undiscovered，out of scope）

2. 需求协商与确认&需求引导同步进行

3. 需求协商以文档的草稿为基础，需求确认需要更加完整的文档确认版本

（二）超出范围的需求

1. 需要对照参考模型才能决定

   (1) 历史上的参考模型以关联图的形式提供——数据流图（DFD），一种流行的结构化建模技术顶层图。虽然 DFD 在 UML 中已被用例图代替，关联图仍然是建立系统边界的很好方法。

（三）需求依赖矩阵（交互矩阵）

1. 需求依赖矩阵的特点

   (1) N×N 的矩阵，表示 N 个需求

   (2) 按一种分类顺序分别在行和列的表头上列出需求标识符

   (3) 矩阵的上三角部分不使用，剩下的单元格表示任何两个需求是否重叠、矛盾或独立

   (4) 矛盾的需求应与用户讨论，也可能要重新陈述这个需求以避免矛盾（矛盾记录保留，对后续开发见）

   (5) 重叠的需求也要重新陈述以消除重叠

   (6) 当需求数目较少，需求依赖矩阵是一种发现需求矛盾和重叠的简单有效的技术。

（四）需求风险和优先级

1. 风险分类

   (1) 技术风险——需求在技术上难以实现

   (2) 性能风险——需求实现后，会延长系统的响应时间

   (3) 安全风险——需求实现后，会破坏系统的安全性

   (4) 数据库完整性风险——需求不容易验证，并且可能导致数据的不一致性

   (5) 开发过程风险——需求要求开发人员采用不熟悉的非常规开发方法，如形式化规格说明方法

   (6) 政治风险——由于内部的政治原因，很难实现需求

   (7) 法律风险——需求可能触犯现行法规或者假定了法律的变更

   (8) 易变性（Volatility）风险——需求很可能再开发过程中不断变化或进化

2. 风险数量决定了项目的可行性（Feasibility），需要对风险进行分析和优先级排序

仅供学习参考

四、需求管理

（一）需求标识与分类

1. 需求以自然语言描述
2. 对需求进行标识分类的 3 种技术
   (1) 唯一标识符——自动生成或 CASE 工具的数据库生成
      ① 数据库生成唯一标识符：最灵活和最不易出错的方法
   (2) 文档层次中的顺序编号——考虑在需求文档中的位置的编号，如“2.4.7”
   (3) 需求分类中的顺序编号——加上一些帮助记忆的标识需求种类的名字赋值
      ① 需求种类分为：功能性需求、数据需求、性能需求、安全需求或其他需求

（二）需求层次

1. 按父子关系建立层次化结构
   (1) 父级需求由子级需求组成，子集需求是父级需求有效的子需求

（三）变更管理

1. 需求变更的原因
   (1) 在软件开发生命周期的任何阶段，需求都可能变更，可能删除已有需求或者增加新的需求
   (2) 变更本身并不会导致困难，但没有管理的变更会更麻烦
   (3) 开发越往前走，需求变更的开销越大
2. 需求变更的实现
   (1) 需要强有力的政策来建立变更请求的文档，估计变更所带来的影响，并实现变更
   (2) 运用的工具：软件配置管理工具（Software Configuration Management Tool）
   (3) 需求可跟踪性：变更管理的一部分，或者说重要部分

仅供学习参考

五、需求业务模型

（一）需求业务模型的定义

1. 在需求确定阶段，所收集需求的高层可视化表示
2. 不属于 UML，是高层可视化模型
3. UML 进行形式化的需求建模在需求规格说明阶段进行

（二）系统范围模型

1. 系统范围的作用
   (1) 需求变更会引起系统范围蠕变
   (2) 当需求中的某些变更不可避免的时候，必须确保请求不会超出项目可接受的范围
2. 界定系统范围的方法
   (1) 通过识别外部实体以及在外部实体和我们系统之间的输入 / 输出数据流来确定
      ① 外部实体：期望从我们这里得到服务或为我们提供服务的其他系统、组织、人员、机器等
   (2) 环境图（Context Diagram）：DFD 的顶层图，是确定系统边界最好的可视化建模方法
   (3) UML 不提供定义系统范围的好的可视化模型

（三）业务用例模型

1. 特点
   (1) 用例（Use case）：整个系统功能的一部分，这一部分一定要是在逻辑上相对完整的功能流程
   (2) 高抽象级别的用例模型
   (3) 为了使 业务过程 被 开发中的信息系统 支持
2. 业务用例
   (1) 标识高层业务进程
   (2) 这样的用例是以某种方式定义一个过程，这个过程完全是从它的实现中抽象出来的
   (3) 业务用例的另一个名称是系统特征（System feature），系统特征在视觉文档中标识。如果存在视觉文档，那么它可以代替业务用例模型
   (4) 在需求规格说明阶段，业务用例被转换成用例，在此阶段标识详细的用例
3. 业务用例图
   (1) 获取的是含有手工业务的业务需求，即系统可能实现不了的业务服务，与用例图区分开。（整个模型代表商业观点，强调组织活动和工作任务，计算机系统不必对此提供支持）
   (2) 焦点：业务过程的体系结构
   (3) 业务用例模型不足以向开发人员确切的表达系统应该做什么
   (4) 业务用例图中的业务参与者有时可以标识环境图的外部实体。这种参与者也被称为次要参与者，它们对于用例是被动的，为了和用例通信，它们需要联合主要参与者。主要参与者是系统的中心，能

仅供学习参考

够与用例主动通信。主要业务参与者通过发送事件来触发用例

(5) 业务用例模型中重要的关系是关联关系，除了关联，一般不鼓励在业务用例之间使用 UML 关系

4. 业务词汇表

(1) 业务词典，一般包含项目陈述中的名词

5. 业务类模型

(1) 和实体类模型差不多，它的抽象级别更高

(2) 业务类模型是 UML 类模型，是标识系统中业务对象的主要类型。

(3) 业务类连接到模型中的 3 个 UML 关系

① 关联（Association）

② 泛化（Generalization）

③ 聚合（Aggregation）

(4) 关联和聚合标识类的实例（对象）之间的语义关系，泛化是类（对象类型）之间的关系

仅供学习参考

六、需求文档

（一）项目准备

1. 项目的目的和范围
2. 业务环境
3. 利益相关者（Stakeholders）
4. 多种解决方案
5. 文档综述

（二）系统服务（功能需求）——占总篇幅的一半及以上

1. 系统范围——使用环境图建模
2. 功能性需求——使用业务用例图建模
3. 数据需求——使用业务类图建模

（三）系统约束（非功能性需求）

1. 界面需求——大概画出界面的外观，给一个界面原型
2. 性能需求，如响应时间、可靠性、吞吐量，用可测量可衡量的数据表示
3. 安全性需求，访问权限
4. 操作性需求，决定系统运行的软硬件环境
5. 政策法律需求
6. 其他约束

（四）项目其他问题

1. 开放问题
2. 初步安排
   (1) 可以使用项目管理软件工具制作标准规划图，如 PRET 或 Gantt 图
3. 初步预算——初步安排的直接结果

（五）附录

1. 业务词汇表（术语、简写词）
2. 业务文档和表格
3. 参考文献

仅供学习参考